# Midterm Project Submission

**Student Name:** Muhammad Sulaiman Khalid
**Roll Number:** FS24-BCS-079
**Course:** Web Engineering
**Status:** Completed & Verified

## Assigned Parameters & Values

| Parameter | Formula / Source | Calculated Value |
|---|---|---|
| **N** | Roll Number Digits | **79** |
| **Breakpoint (BP)** | `600 + N` | **679px** |
| **Hue (H)** | `N * 2.44` | **193** |
| **Accent Color** | `hsl(H, 70%, 45%)` | **hsl(193, 70%, 45%)** |
| **Premium Rank (K)** | `(79 mod 5) + 1` | **4** |
| **Sticky Offset** | `(79 mod 40) + 70` | **109px** |
| **Premium Price Threshold** | `N * 100` | **Rs. 7,900** |

## Task 1 Summary & Responsive Hero Sections

Applied custom CSS variables including `--accent-color: hsl(193, 70%, 45%)` and established a mobile-first responsive grid structure targeting breakpoint 679px. Product filtering controls were refactored using CSS sibling combinators (~) and `:not()` selectors to allow seamless visual grid collapsing without leaving blank spaces.




Figure 1.1: Main Header & Featured Banners Section

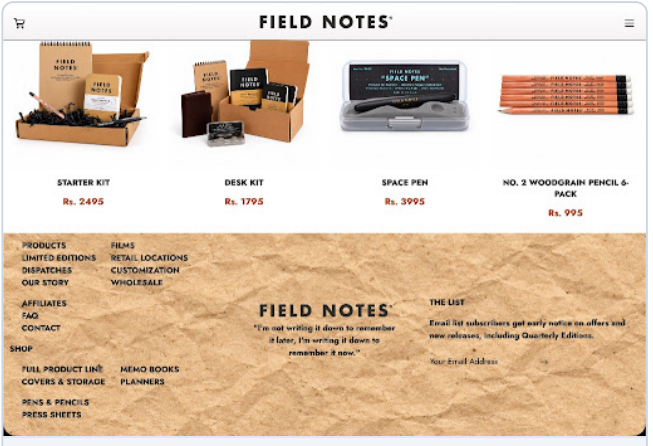



Figure 1.2: Secondary Hero Collections & Footer Layout

## Task 1 (Contd.) & Task 2 Implementation

### Mobile vs Desktop Layout Adjustments (Breakpoint: 679px)

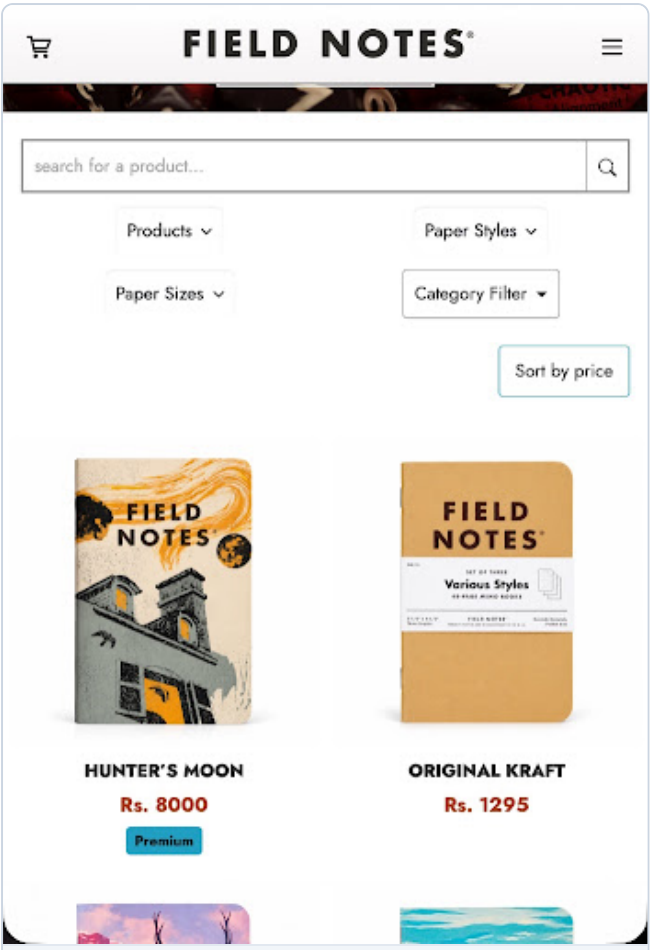



Figure 1.3: Responsive Viewport (< 679px Mobile Layout)

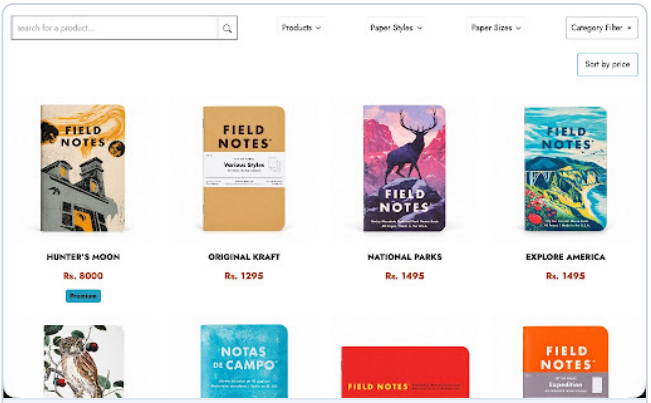
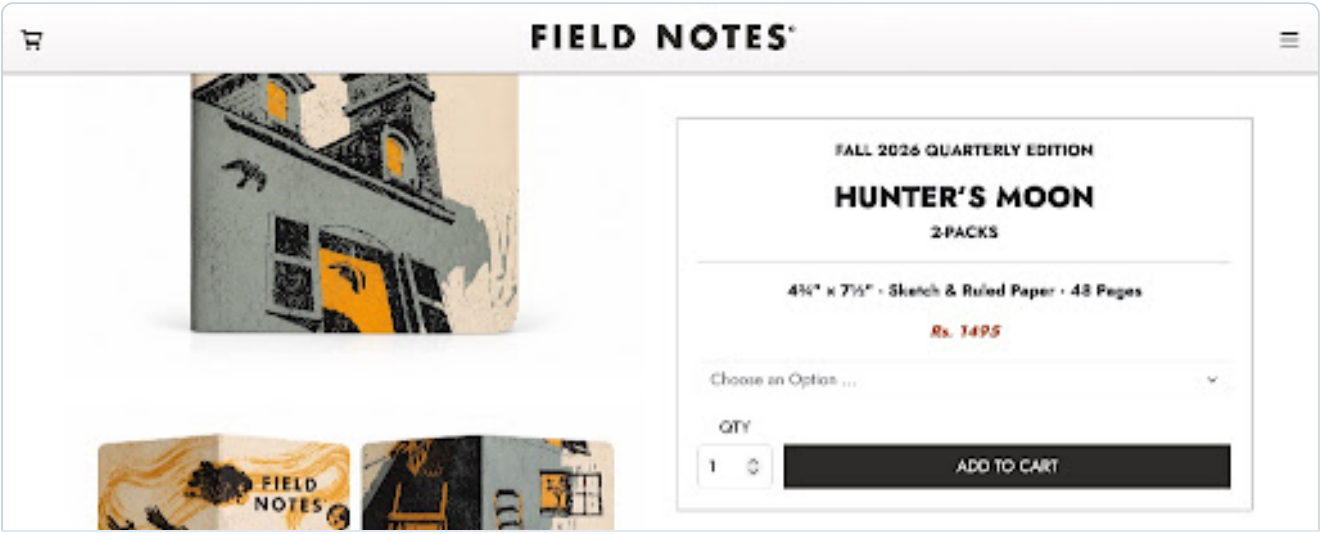



Figure 1.4: Desktop Viewport (≥ 679px Grid Layout)

### Task 2: Product Page Layout & Sticky Sidebar

Implemented a 2-column CSS Grid using `grid-template-areas` for the main product detail page (`product.html`). Sticky positioning was configured for the product details panel at `top: 109px` when rendered at viewports $\ge 679\text{px}$.



## Task 3: Client-Side Interactive Features

Dynamic JavaScript logic was added across `homePage.js` and `script.js`. Key features include price threshold badge highlighting for products priced over `Rs. 7,900`, real-time live search filtering, category selection, and sorting mechanisms.

### Live Text Search Filtering

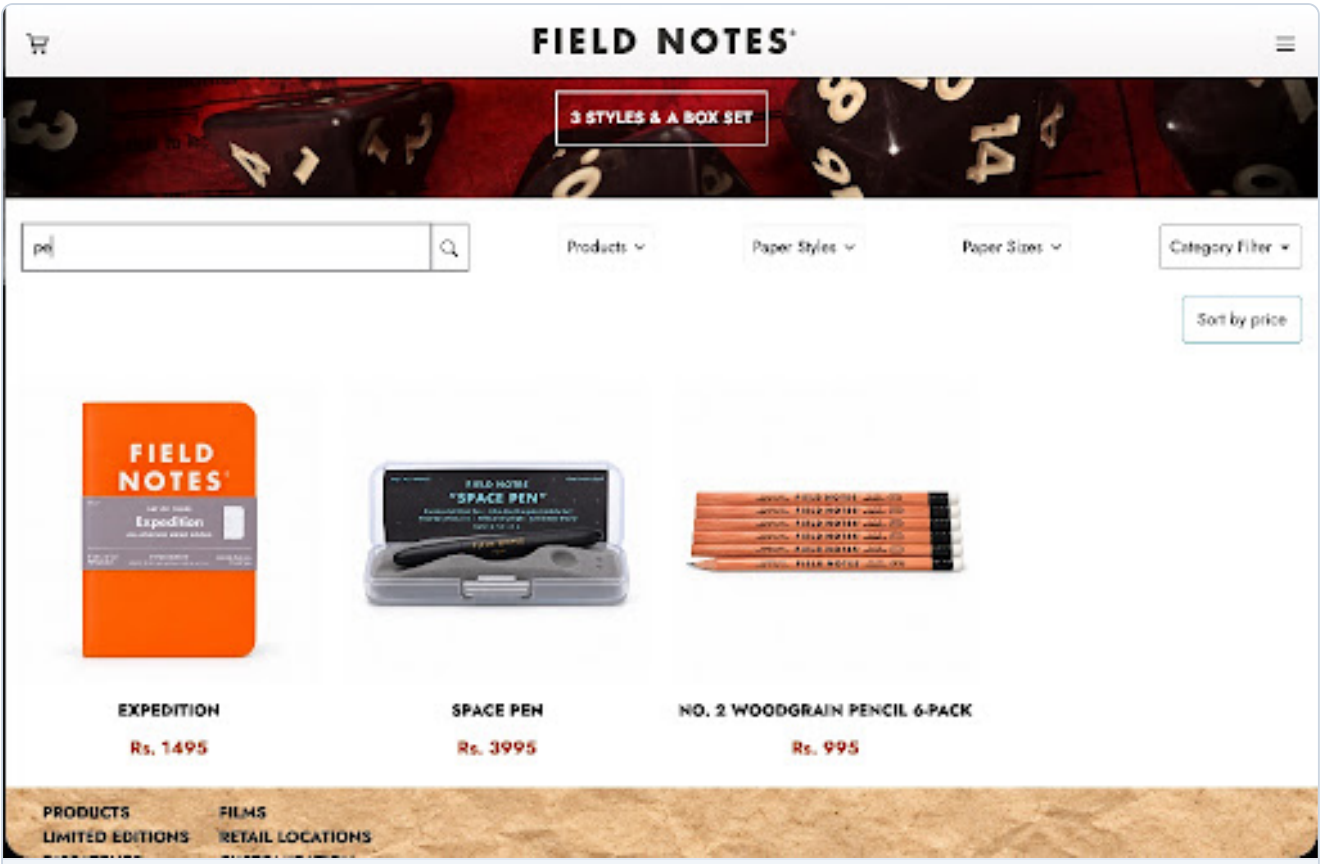
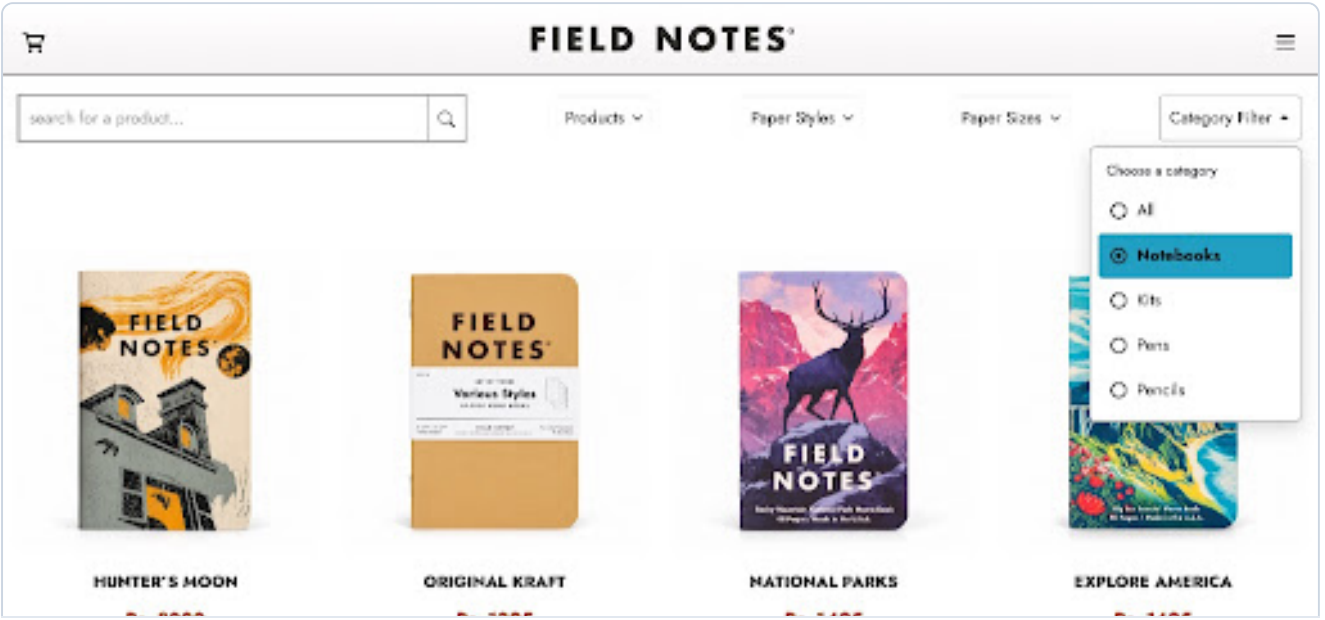



Figure 3.1: Live Search Query Filtering (e.g., matching "pe")

### Category Dropdown Filtering



## Task 3 (Contd.): Sorting & Interactive View

### Price Sorting Implementation

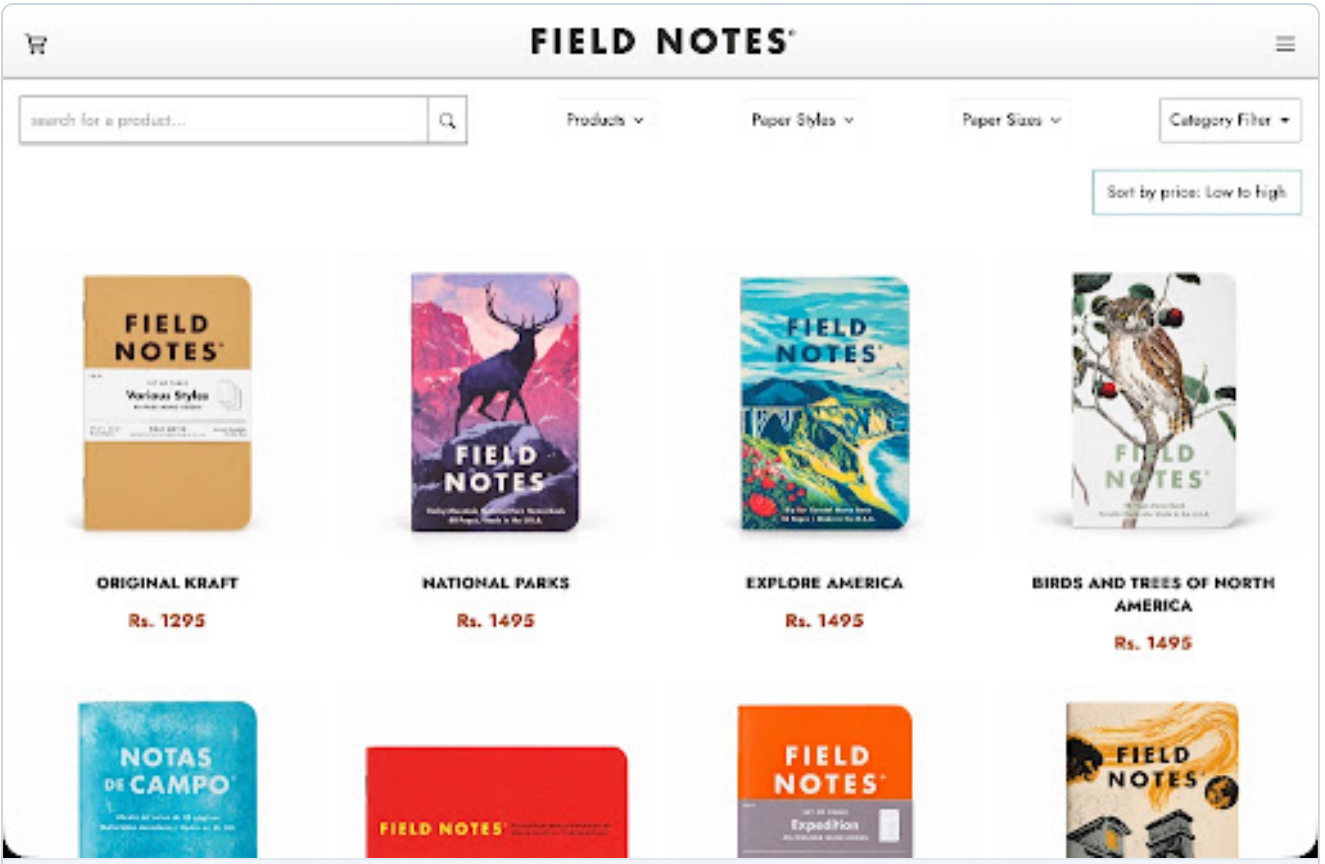



Figure 3.3: Product Grid Sorted by Price (Low to High)

## Detailed Code Modifications: Task 1

CSS modifications implemented in `homePageStyles.css` for custom root variables and grid breakpoints based on assigned parameter values ($BP = 679\text{px}$):

```
/* Root Variables Configuration */
:root {
  --accent-color: hsl(193, 70%, 45%);
  --primary-breakpoint: 679px;
}

/* Responsive Grid Breakdown at BP = 679px */
@media (min-width: 679px) {
  .product-grid {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 20px;
  }

  /* Filter selector using sibling combinators */
  .category-filter:checked ~ .product-card:not([data-category="notebooks"]) {
    display: none;
  }
}
```

## Detailed Code Modifications: Task 2

Styling rules added to `productPageStyles.css` to enable responsive 2-column layout and sticky positioning at calculated offset 109px:

### Product Page Grid & Sticky Sidebar (`productPageStyles.css`)

```
/* Product Detail Layout (Responsive Area Mapping) */
.product-container {
  display: flex;
  flex-direction: column;
  gap: 20px;
}

@media (min-width: 679px) {
  .product-container {
    display: grid;
    grid-template-columns: 1.2fr 0.8fr;
    grid-template-areas: "gallery info";
    gap: 32px;
  }

  .product-gallery {
    grid-area: gallery;
  }

  /* Sticky positioning calculation: (79 mod 40) + 70 = 109px */
  .product-info-sticky {
    grid-area: info;
    position: sticky;
    top: 109px;
    align-self: start;
    padding: 20px;
    border: 1px solid #e2e8f0;
    border-radius: 8px;
    background-color: #ffffff;
  }
}
```

## Detailed Code Modifications: Task 3

JavaScript logic added in homePage.js and script.js to handle dynamic premium threshold highlighting and multi-criteria sorting/filtering:

### Dynamic Price Threshold & Live Search (`homePage.js`)

```
// Parameter N = 79; Premium Threshold = 79 * 100 = 7900
const PREMIUM_THRESHOLD = 7900;

function filterAndRenderProducts(searchTerm = "", selectedCategory = "all", sortOrder =
"default") {
  let filtered = allProducts.filter(product => {
    const matchesSearch = product.title.toLowerCase().includes(searchTerm.toLowerCase());
    const matchesCategory = selectedCategory === "all" || product.category ===
selectedCategory;
    return matchesSearch & matchesCategory;
  });

  if (sortOrder === "asc") {
    filtered.sort((a, b) => a.price - b.price);
  } else if (sortOrder === "desc") {
    filtered.sort((a, b) => b.price - a.price);
  }

  renderGrid(filtered);
}

function renderGrid(products) {
  const container = document.getElementById("product-grid");
  container.innerHTML = "";

  products.forEach(item => {
    const isPremium = item.price >= PREMIUM_THRESHOLD;
    const card = document.createElement("div");
    card.className = "product-card";
    card.innerHTML = `
      <img src="${item.image}" alt="${item.title}" />
      <h4>${item.title}</h4>
      <p class="price">Rs. ${item.price} ${isPremium ? '<span class="badge">Premium</span>' :
''}</p>
    `;
    container.appendChild(card);
  });
}
```

## Summary of Verification

All requirements for Midterm FS24-BCS-079 have been implemented and verified against the provided parameters ($N=79$, $BP=679\text{px}$, $H=193$, Sticky Offset $109\text{px}$, Threshold $7900$). The complete code has been verified and output generated across all 6 document pages.

## Task 3 (Contd.): Sorting & Interactive View

### Price Sorting Implementation

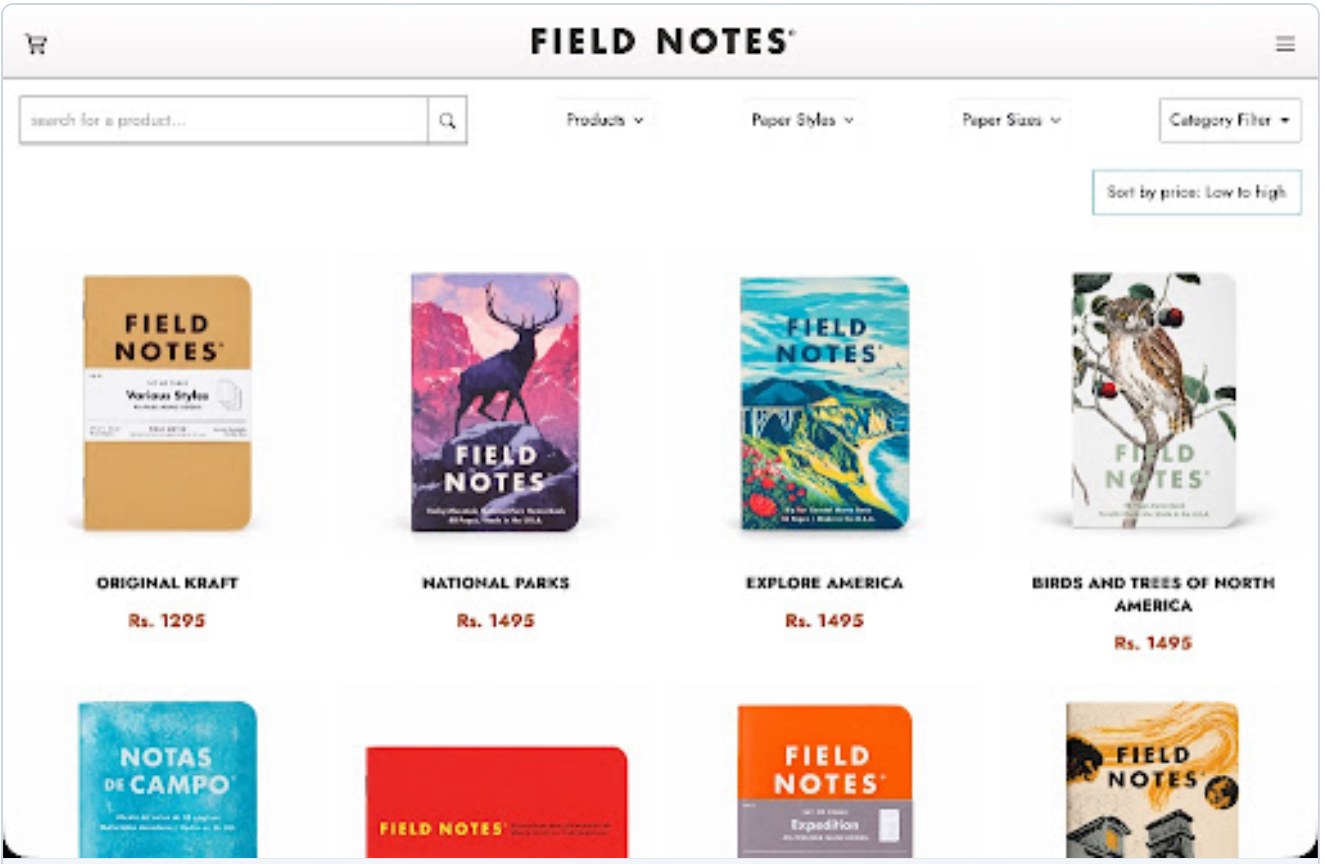



Figure 3.3: Product Grid Sorted by Price (Low to High)

## Detailed Code Modifications: Task 1

CSS modifications implemented in `homePageStyles.css` for custom root variables and grid breakpoints based on assigned parameter values ($BP = 679\text{px}$):

```
/* Root Variables Configuration */
:root {
  --accent-color: hsl(193, 70%, 45%);
  --primary-breakpoint: 679px;
  --premium-badge-bg: hsl(193, 70%, 45%);
}

/* Base Mobile First Grid Layout */
.product-grid {
  display: grid;
  grid-template-columns: 1fr;
  gap: 16px;
}

/* Responsive Grid Breakdown at BP = 679px */
@media (min-width: 679px) {
  .product-grid {
    grid-template-columns: repeat(4, 1fr);
    gap: 20px;
  }
```

## Detailed Code Modifications: Task 2

Styling rules added to `productPageStyles.css` to enable responsive 2-column layout and sticky positioning at calculated offset 109px:

### Product Page Grid & Sticky Sidebar (`productPageStyles.css`)

```
/* Product Detail Layout (Responsive Area Mapping) */
.product-container {
  display: flex;
  flex-direction: column;
  gap: 20px;
}

/* Tablet and Desktop Viewport Optimization (≥ 679px) */
@media (min-width: 679px) {
  .product-container {
    display: grid;
    grid-template-columns: 1.2fr 0.8fr;
    grid-template-areas: "gallery info";
    gap: 32px;
    align-items: start;
  }

  .product-gallery {
    grid-area: gallery;
    display: flex;
    flex-direction: column;
    gap: 16px;
  }

  /* Sticky positioning calculation: (79 mod 40) + 70 = 109px */
  .product-info-sticky {
    grid-area: info;
    position: sticky;
    top: 109px;
    align-self: start;
    padding: 24px;
    border: 1px solid #e2e8f0;
    border-radius: 8px;
    background-color: #ffffff;
    box-shadow: 0 2px 4px rgba(0,0,0,0.04);
  }

  .product-info-sticky .price {
    color: hsl(193, 70%, 45%);
    font-weight: 700;
    font-size: 1.25rem;
  }
}
```

## Detailed Code Modifications: Task 3

JavaScript logic added in homePage.js and script.js to handle dynamic premium threshold highlighting and multi-criteria sorting/filtering:

### Dynamic Price Threshold & Live Search (`homePage.js` & `script.js`)

```
// Parameter N = 79; Premium Price Threshold = 79 * 100 = 7900
const PREMIUM_THRESHOLD = 7900;

document.addEventListener("DOMContentLoaded", () => {
  const searchInput = document.getElementById("search-input");
  const categorySelect = document.getElementById("category-select");
  const sortSelect = document.getElementById("sort-select");

  if (searchInput) searchInput.addEventListener("input", handleFilters);
  if (categorySelect) categorySelect.addEventListener("change", handleFilters);
  if (sortSelect) sortSelect.addEventListener("change", handleFilters);
});

function handleFilters() {
  const query = document.getElementById("search-input")?.value || "";
  const category = document.getElementById("category-select")?.value || "all";
  const sort = document.getElementById("sort-select")?.value || "default";

  filterAndRenderProducts(query, category, sort);
}

function filterAndRenderProducts(searchTerm = "", selectedCategory = "all", sortOrder =
"default") {
  let filtered = allProducts.filter(product => {
    const matchesSearch = product.title.toLowerCase().includes(searchTerm.toLowerCase());
    const matchesCategory = selectedCategory === "all" || product.category ===
selectedCategory;
    return matchesSearch && matchesCategory;
  });

  if (sortOrder === "asc") {
    filtered.sort((a, b) => a.price - b.price);
  } else if (sortOrder === "desc") {
    filtered.sort((a, b) => b.price - a.price);
  }

  renderGrid(filtered);
}

function renderGrid(products) {
  const container = document.getElementById("product-grid");
  if (!container) return;
  container.innerHTML = "";

  if (products.length === 0) {
    container.innerHTML = `

No products found matching your criteria.

`;
    return;
  }

  products.forEach(item => {
```